Speaking of transactions, what is a *database transaction*? Rails developers may be less familiar with database transactions. Here are the basics.

Transactions can be explicitly created, but are usually implicit based on the statements you're running. Transactions help facilitate making multiple modifications to structure or data as one unit.

Try out Transactional DDL a bit with an experiment. Try making a DDL change and use an explicit transaction that you create.

Inside psql, type `BEGIN;` to open your transaction.

Inside the transaction create a DDL change like adding an index, like you did earlier. When you aren't using an explicit transaction, an implicit transaction is created for the operation.

When you use an explicit transaction you have "transaction control." With transaction control you may decide to roll back the transaction. After typing the `CREATE INDEX` statement and a semi-colon to end the line, on a new line type `ROLLBACK;`.

The `ROLLBACK` keyword rolls back a transaction, which means it's not applied or saved to the database. When this happens, thanks to the Transactional DDL feature the index creation is not applied. From psql, confirm that the index does not exist.

The following statements show this scenario where an index is added inside a transaction that can be committed or rolled back:

sql/transactional_ddl_changes.sql

```
BEGIN;

CREATE INDEX test_index ON tbl (col);

ROLLBACK;
```

Now that you've seen Transactional DDL, it may be tempting to experiment using your production database. Is this a good idea?

Exploring and Experimenting Safely In Production

Most companies have a staging environment that's running the application with a PostgreSQL database separate from production.

A staging environment is a great place to perform some kinds of experimentation. Take advantage of this if it's available to you!

Write down the operations you're performing in a ticket tracking system and share it with a teammate who can double check it. Create a backup before